

Umbrella Notes — Module 6

Algorithms + Software Engineering | shared core for Coal India MT (Systems), IBPS SO IT (SPL-XVI), DSSSB TGT CS (DOE)

How to read this module

This is **full teaching notes** — learn directly from it. Depth is **balanced across all three exams** (middle depth by default: enough for IBPS SO, DSSSB TGT, and general PSU/Govt CS-IT-Systems exams).

■ **Green box** = how PSU/Govt exams actually test the topic (a reference lens, not a limit).

↑ **Amber box** = extra GATE-core depth needed *only* if targeting Coal India MT (or GATE-style PSUs). Skip for IBPS/DSSSB.

■ **Purple box** = a coverage warning (something kept light, or where an exam needs more/less).

Small sub-topics are folded under umbrella headings but never dropped. Topic-wise Q&A with explanations comes as a separate set after your walkthrough.

0 What this module covers

The final shared-core module bundles two mid-size topics. **Algorithms** completes the complexity-analysis thread pointed to from Modules 4 and 5 — how we measure efficiency and the standard techniques for searching, sorting, and solving problems. **Software Engineering** zooms out to how large software is planned, built, and maintained. Together they close the Tier-1 umbrella.

- **Part A — Algorithms:** complexity & asymptotic notation, searching, sorting, design techniques (greedy / DP / divide-and-conquer), graph algorithms.
- **Part B — Software Engineering:** SDLC & process models, requirements, design concepts, testing, and maintenance.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Algorithms: high-yield MCQs are **time complexity of sorting/searching**, **best/worst cases**, and **which technique** a named algorithm uses. Software Engineering: **SDLC phase order**, **process-model features** (Waterfall vs Agile vs Spiral), and **testing types**.

Software Engineering is explicitly listed in IBPS SO IT and DSSSB (DOE); it is light but reliable marks — pure conceptual recall.

PART A Algorithms

A.1 What makes a good algorithm; complexity

An **algorithm** is a finite, well-defined sequence of steps that solves a problem. We compare algorithms by **time complexity** (steps vs input size n) and **space complexity** (extra memory vs n), expressed with **asymptotic notation**:

Notation	Bounds	Meaning
Big-O (O)	Upper bound	Worst-case growth ('no worse than')
Omega (Ω)	Lower bound	Best-case growth ('at least')
Theta (Θ)	Tight bound	Exact growth (upper = lower)

Growth-rate order (slowest-growing = best): $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$. Memorising this ranking answers many 'which is more efficient' questions instantly.

A.2 Searching

Algorithm	Requires	Best	Average / Worst
Linear search	Nothing	$O(1)$	$O(n) / O(n)$
Binary search	Sorted array	$O(1)$	$O(\log n) / O(\log n)$

Binary search repeatedly halves a **sorted** array (divide-and-conquer) — the classic $O(\log n)$. The 'sorted' precondition is a favourite trap.

A.3 Sorting — learn this table cold

Algorithm	Best	Average	Worst	Stable?
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Sorting complexities are the single most-tested algorithms MCQ. Nail: Merge/Heap = $O(n \log n)$ always; Quick = $O(n \log n)$ avg but $O(n^2)$ worst; Bubble/Selection/Insertion = $O(n^2)$.

Know the **technique** each uses: Merge & Quick = **divide-and-conquer**; and which are **stable** (Bubble, Insertion, Merge).

A.4 Algorithm design techniques

Technique	Idea	Examples
Divide & conquer	Split into subproblems, solve, combine	Merge sort, Quick sort, binary search
Greedy	Make the locally-best choice at each step	Dijkstra, Prim, Kruskal, Huffman
Dynamic programming	Solve overlapping subproblems, store results	Knapsack, LCS, Floyd-Warshall, Fibonacci
Backtracking	Try options, undo on failure	N-Queens, maze, sudoku

Greedy vs DP: greedy commits to one choice and never reconsiders (fast, not always optimal); DP explores overlapping subproblems and stores answers (optimal, more memory). A common comparison MCQ.

A.5 Graph algorithms (build on Module 4)

- **Traversals:** **BFS** (queue, shortest path in unweighted graphs) and **DFS** (stack/recursion, cycle detection, topological sort).
- **Minimum Spanning Tree (MST):** **Prim** and **Kruskal** (both greedy) — connect all vertices with minimum total edge weight.
- **Shortest paths:** **Dijkstra** (greedy, non-negative weights, single source), **Bellman-Ford** (handles negative weights), **Floyd-Warshall** (all-pairs, DP).

↑ Go deeper — GATE-core (Coal India MT)

Coal India / GATE: solve **recurrences** (Master theorem, substitution), trace algorithms step-by-step, prove correctness/optimality, and run Dijkstra/Prim/Kruskal/Floyd-Warshall on given graphs. Also amortized analysis and NP-completeness basics (P vs NP, reductions). IBPS/DSSSB stop at complexities + which-technique recall.

PART B Software Engineering

B.1 The SDLC

The **Software Development Life Cycle (SDLC)** is the structured sequence of phases for building software. The standard order (memorise it):

- **1. Requirement analysis** — gather & document what the software must do (produces the SRS).
- **2. Design** — plan architecture, modules, data, interfaces.
- **3. Implementation / Coding** — write the actual code.
- **4. Testing** — verify it works & meets requirements.
- **5. Deployment** — release to users.
- **6. Maintenance** — fix bugs & enhance after release (usually the longest, costliest phase).

■ The SRS document

The **Software Requirements Specification (SRS)** is the output of the requirements phase — the agreed statement of what the system will do. Requirements split into **functional** (what it does) and **non-functional** (how well: performance, security, usability).

B.2 Process models

Model	Idea	Best when...
Waterfall	Linear, one phase fully before the next	Requirements are fixed & clear
Iterative / Incremental	Build in repeated cycles, adding features	Requirements evolve
Spiral	Iterative + heavy risk analysis each loop	Large, high-risk projects
Prototype	Build a quick model, refine from feedback	Requirements are unclear
Agile	Small, fast, collaborative iterations (sprints)	Changing needs, quick delivery
V-Model	Waterfall with a testing phase paired to each dev phase	Verification-critical systems

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Waterfall vs Agile is the central comparison: Waterfall = rigid, sequential, no going back; Agile = flexible, iterative, customer-collaborative (Scrum, sprints).

MCQs ask **which model** suits a scenario (fixed requirements → Waterfall; unclear → Prototype; high-risk → Spiral) and **which SDLC phase** a task belongs to.

B.3 Design concepts & testing

- **Cohesion** (how focused one module is — want it **high**) vs **Coupling** (interdependence between modules — want it **low**). 'High cohesion, low coupling' is the goal — a very common MCQ.
- **Modularity & abstraction** — breaking a system into independent, manageable parts.

Testing type	What it checks
Unit testing	A single module/function in isolation
Integration testing	Modules working together
System testing	The complete integrated system
Acceptance testing	Meets user requirements (done by/for the client)
Black-box	Functionality without seeing code
White-box	Internal logic/code paths

Verification vs validation: verification = 'are we building it right?' (meets spec); validation = 'are we building the right thing?' (meets user needs). Frequently confused — and frequently asked.

↑ Go deeper — GATE-core (Coal India MT)

Coal India / GATE-style: effort/cost estimation (**COCOMO**), **function points**, software metrics, **cyclomatic complexity**, and detailed testing strategies. These are heavier than IBPS/DSSSB need — which is mostly SDLC + model + testing-type recall.

■ Coverage note / warning

Software Engineering in these exams rewards **breadth of recall over depth**. This module covers the tested surface; don't over-invest in derivations — the marks are in clean definitions and comparisons (SDLC order, model features, cohesion/coupling, testing types, verification vs validation).

Rapid recap — highest-yield facts

Area	Nail these
Complexity	Big-O(upper/worst), Ω (lower/best), Θ (tight); order $O(1) < \log n < n < n \log n < n^2 < 2^n < n!$
Searching	Linear $O(n)$; Binary $O(\log n)$ but needs SORTED array
Sorting	Merge/Heap $O(n \log n)$ always; Quick avg $O(n \log n)$ /worst $O(n^2)$; Bubble/Selection/Insertion $O(n^2)$; stable = Bubble/Insertion/Merge
Techniques	Divide&conquer(merge/quick/binary), Greedy(Dijkstra/Prim/Kruskal), DP(knapsack/LCS/Floyd-Warshall), Backtracking(N-Queens)
Graphs	BFS(queue)/DFS(stack); MST Prim&Kruskal(greedy); shortest path Dijkstra(greedy)/Bellman-Ford(neg)/Floyd-Warshall(all-pairs DP)
SDLC	Requirements→Design→Coding→Testing→Deployment→Maintenance(longest); SRS = requirements output
SE models	Waterfall(rigid/fixed reqs) vs Agile(iterative/flexible); Spiral(risk); Prototype(unclear reqs); high cohesion + low coupling; verification vs validation

✓ Tier-1 shared core complete

This finishes the six-module umbrella: **Computer Networks, Operating Systems, DBMS & SQL, Programming & Data Structures, Digital Logic + COA, and Algorithms + Software Engineering** — the ~70-80% of syllabus shared across all three exams.

Flag any page/section that felt shaky and I'll go deeper; then the **topic-wise Q&A sets** follow for live practice. After that we move to the **Tier-2 / exam-wise PDFs** (Coal India TOC/Compiler; IBPS Cloud/Cyber + descriptive; DSSSB Java/.NET/web/graphics/MIS + general track).

Coal India target? The amber ↑ **GATE-core** boxes here — recurrences/Master theorem, algorithm tracing, graph-algorithm runs, COCOMO/cyclomatic complexity — are the extra depth.